2024 Twelfth International Symposium on Computing and Networking Workshops (CANDARW) | 979-8-3315-0534-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/CANDARW64572.2024.00016

An Improved Method for Detecting EDoS Attacks in the Cloud With Hyperparameter Optimization and Metaheuristic Algorithms

Alessandro Cordeiro de Lima[1], Eduardo A. P. Alchieri[1], Jacir L. Bordim[1], João J. C. Gondim[2]
[1]Department of Computer Science, [2]Department of Electrical Engineering
University of Brasília, Brasília-DF, Brazil

***Abstract*—In recent years, cloud computing has gained significant popularity and adoption. As cloud infrastructure expands, vulnerabilities to attacks that exploit cloud services to drain paid resources also increase. A new form of attack, known as Economic Denial of Sustainability (EDoS) or Fraudulent Resource Consumption (FRC), is emerging as a major concern. This attack disrupts the cloud payment model by gradually escalating resource demand, leading to severe financial strain for both customers and service providers. EDoS attacks are classified as low-rate denial of service (LDoS) attacks, characterized by their covert nature and the challenges they pose to traditional detection methods. This paper introduces a methodology for detecting EDoS attacks by combining the Synthetic Minority Oversampling Technique (SMOTE) and the Edited Nearest Neighbor Rule (ENN) with Random Forest (RF) and XGBoost (XGB) classifiers. These classifiers are optimized using nature-inspired metaheuristics and conventional hyperparameter tuning methods, such as Random Search and Bayesian Search. Our comparative approach demonstrated improved performance, reduced computational costs, and decreased parameter search time. Notably, the application of the Bat Algorithm (BA) to optimize the XGBoost classifier showed superior results in identifying EDoS attacks, achieving higher accuracy (99.53%) and a lower error rate (0.46%) compared to other algorithms studied, including standard Random and Bayesian search methods.**

***Index Terms*—EDoS, ML, SMOTE-ENN, Cloud Computing.**

I. Introduction

Cloud computing has emerged as a critical paradigm for the IT sector, offering on-demand services that are cost-effective, flexible, and scalable [21]. However, this infrastructure introduces novel security challenges, including Economic Denial of Sustainability (EDoS) attacks [2]. EDoS attacks fraudulently deplete computing resources, causing financial losses to clients and degrading platform performance by exploiting the pay-as-you-go (PAYG) payment model.

Early detection of EDoS attacks is crucial for ensuring the security and reliability of cloud services. However, these attacks are challenging to detect using conventional methods due to their covert nature. Additionally, the absence of specific datasets for EDoS attacks and the necessity of hyperparameter optimization in Machine Learning (ML) algorithms present further obstacles in detecting EDoS attacks [24].

EDoS attacks are categorized as a type of Low-rate Denial of Service (LDoS) because they are non-volumetric and mimic legitimate traffic [31]. In fact, while DoS (Denial of Service) and DDoS (Distributed DoS) attacks aim to interrupt cloud services with a large volume of malicious requests, LDoS attacks are more sophisticated and difficult to detect due to their low flow rate and stealthy behavior. They evade defense mechanisms for high-speed DDoS attacks by sending malicious requests in brief intervals at low rates. These characteristics make EDoS attacks particularly insidious, as they can persist undetected for extended periods, causing substantial financial damage before being identified [31]. Several simulations and real attacks have demonstrated the offensive power of EDoS attacks [13], [19]. For instance, Shah et al. [25] executed an experiment that generated 1,000 requests per second for a service on Amazon CloudFront for 30 days, resulting in an additional cost of US$ 42,000. Similarly, in [20], it is reported an unexpected charge of US$ 1,300 in one day, due to a Denial of Wallet (DoW) attack, which is a type of EDoS. Such cases underscore the significant financial impact that EDoS attacks can have on cloud services.

This study introduces an EDoS detection model that combines the Synthetic Minority Over-sampling Technique (SMOTE) and the Edited Nearest Neighbors (ENN) rule with Random Forest (RF) and XGBoost (XGB) classifiers. The classifiers are optimized using both nature-inspired metaheuristic algorithms and standard hyperparameter optimization approaches. The suggested methodology focuses on enhancing classification accuracy and computational efficiency. The current proposals (e.g., [3], [27]) to detect EDoS are evaluated using datasets composed by many different attacks (e.g., DoS, backdoor and EDoS) and do not differentiate these attacks for detection. Consequently, these solutions are evaluated considering also the detection of attacks whose characteristics different from EDoS attacks. The first contribution of this work is the characterization of the attacks contained in a well-known dataset (UNSW-NB15 [18]), separating EDoS from other attacks. This characterization is derived from the research conducted by Zhijun et al. [31] and enabled us to assess our approach specifically in the detection of EDoS attacks, which are typically more challenging to identify.

The additional contributions of this work are as follows: *(i)* the use of the SMOTE-ENN technique to balance the resulting dataset for EDoS attacks, *(ii)* the optimization of hyperparameters for the ML algorithms (RF and XGB) using

2832-1324/24/$31.00 ©2024 IEEE
DOI 10.1109/CANDARW64572.2024.00016

nature-inspired metaheuristic algorithms such as Bat algorithm (BA), Hybrid bat algorithm (HBA), Hybrid self-adaptable bat algorithm (HSABA), Firefly algorithm (FA), Grey wolf optimizer algorithm (GWO) and Genetic algorithms (GA), and *(iii)* a comparative assessment of these metaheuristic methods against conventional hyperparameter optimization techniques like Random Search (RS) and Bayesian Search (BS). In this study, the SMOTE-ENN technique is employed to balance the dataset and address class imbalance, which is crucial for improving the performance of ML algorithms. The use of metaheuristic algorithms, for hyperparameter optimization, helps achieve a balance between computational cost and performance. These algorithms are inspired by natural processes and have proven effective in various optimization tasks [22]. By optimizing the hyperparameters of the RF and XGB algorithms, the study aims to enhance classification accuracy and computational efficiency.

The comparative assessment of optimization methods involves assessing the performance, computational cost, and search duration of metaheuristic methods in comparison to conventional procedures. Metrics such as recall, accuracy, F1-score, error rate, and execution time are employed to assess the efficiency of various approaches. The findings demonstrate that metaheuristic methods, such as Bat algorithm (BA), surpass conventional techniques like Random Search (RS) and Bayesian search (BS) in terms of accuracy and recall rates, while also reducing computational cost and execution time. These results show that the proposed methodology provides superior classification accuracy and computational efficiency compared to conventional techniques, enhancing the security and dependability of cloud computing. This contribution is significant for cloud service providers and users, as it offers a more effective means of detecting EDoS attacks, possibly leading to better mitigation, thus improving the overall security and reliability of cloud-based services.

The remainder of this paper is organized as follows. Section II surveys related works. Section III presents the materials and methods used in this work, detailing the dataset used to evaluate our proposal. Section IV reports on our experimental evaluation. Finally, Section V concludes the paper.

II. Related Works

This section presents the most relevant research and solutions to detect/mitigate Economic Denial of Sustainability (EDoS) and Fraudulent Consumption of Resources (FRC) attacks found in the literature.

A. Mathematical and statistical models

Agrawal and Tapaswi [1] introduced a method to identify and counteract EDoS attacks using two mathematical models: the Poisson distribution for service rate and the exponential distribution for spurious traffic. The authors did not assess performance indicators using false positive (FPR) and false negative (FNR) rates. Bhushan and Guptab [4] proposed a hypothesis test approach to FRC attacks on cloud services that is based on F-statistics and Turing tests. The results indicated that the detection rates were modest. Also, the evaluation considered outdated datasets (DARPA 99 and CAIDA 2007).

B. Deep learning and machine learning models

Aldhyani and Alkahtani [2] proposed an EDoS detection model that evaluated several machine learning methods, including SVM, KNN, RF, CNN, and LSTM. Nevertheless, the approach fails to identify low-rate attacks. In a separate study, Hossain and Islam [12] employed a variety of ML techniques to detect EDoS attacks, with a particular emphasis on LightGBM. In order to identify the optimal parameters, statistical methodologies were implemented. The study did not provide information regarding the computational cost and response time parameters of the algorithms.

EDoS attacks were also investigated using the Stack Deep Polynomial Network (SDPN) model [27], which is a deep learning method used to acquire precise representations of multivariate time sequences. Nevertheless, some critical performance parameters, including false positive (FPR) and negative rates (FNR), were not assessed. Alsaadi et al. [3] employed RNN-LSTM algorithms to detect EDoS attacks through multi-classification. Nevertheless, their findings suggest that the detection efficiency is modest.

C. Models that employ meta-heuristic algorithms and machine learning

An approach to detecting FRC attacks in the cloud was proposed by Rubai [24]. This approach includes the use of discrete Wavelet transform, adaptive deer hunting optimization algorithm (ADHOA), and recurrent neural networks (RNN). The EDoS-Dome [23] is another system that has been developed to combat EDoS attacks. This system includes user authentication, data obfuscation, load balancing, and user data classification. The computational cost of the algorithms and the duration of the hyperparameters search were not discussed in the results.

Finally, Lima et al. [7] also investigated EDoS attacks detection using meta-heuristic and machine learning algorithms like Random Forest. However, the detection time was not assessed and, as the other works discussed in this section, the performance is analyzed considering the detection of many different attacks (e.g., DoS) instead of only EDoS attacks.

III. Materials and Methods

This section details the dataset used, its pre-processing and adjustments, as well as the algorithms, types of hyperparameters, and the metrics used to evaluate our proposal.

A. Dataset

The dataset UNSW-NB15 [18], which is extensively utilized in academic research, has been chosen to evaluate the effectiveness of methods in detecting EDoS attacks [3], [27]. The dataset is constructed using synthetic data and includes local and cloud attacks. It encompasses 9 attack models, namely Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The data is collected from 49 resources using Argus and Bro-IDS software.

Given that the UNSW-NB15 dataset encompasses many types of attacks in addition to EDoS, we undertook a meticulous process of manually selecting and identifying the specific attacks included in this dataset. This identification was established using the attack characteristics specified by Zhijun et al. [31]. We identified several types of attacks, including ICMP Flood, TCP SYN Flood, DNS Flood, UDP Flood, HTTP Flood, and also EDoS attacks with LDoS characteristics (e.g., Slowloris, Database API Request, XML External Entities - XXE, BREACH attack, Apache HTTP Server Range Header Memory Exhaustion). For these EDoS attacks, we added a new label "EDoS", creating a new type of attack in the dataset. The new set of attacks is shown in Figure 1. This new dataset, called EDoS-UNSW-NB15, is available on Github [15]. This classification enabled us to specifically assess our proposal's effectiveness in detecting EDoS attacks, which are typically more challenging to identify.

B. Preprocessing

The flows contained in the dataset were preprocessed before computation, starting by cleaning and removing null values. From 49 attributes and 257,673 flows, this processing maintained 45 attributes and 256,890 flows, distributed in ten attacks and normal flows (Figure 1). After, the following techniques were applied to prepare the dataset [10].

1) One-Hot Encoding: One-hot coding was applied to transform the categorical variables "protocol", "service" and "state" into numerical values, facilitating the detection of attacks by the classifier algorithms.

2) Feature Scaling: Feature scaling is the practice of normalizing or standardizing independent variables in machine learning. This is done to prevent larger values from having more weight than smaller ones. In this study, min-max scalers are used to evaluate the performance of machine learning models. Furthermore, these scalers also help improve the performance and convergence of metaheuristic algorithms by ensuring that all resources contribute equally to the optimization process. In the equation III.1, *X* is the original value, *X_min* and *X_max* are the minimum and maximum values of the variable, respectively. This equation adjusts the data to the range [0, 1] or [-1, 1],

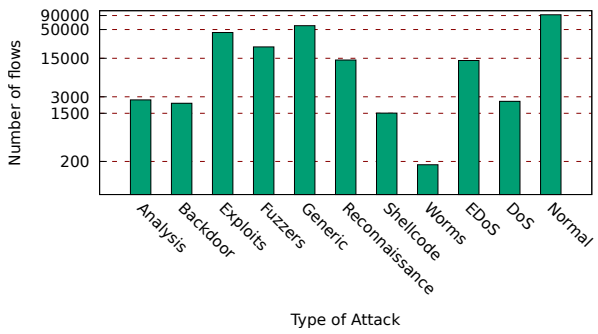

Fig. 1: The dataset contains ten attacks and normal flows.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}. \quad \text{(III.1)}$$

3) Data balancing: It is important to observe that out of the 256,890 flows stored in the dataset, only 13,659 were identified and classified as EDoS attacks (Figure 1). The remaining 243,231 flows, all were categorized as normal flows, which encompasses normal flows as well as various sorts of attacks, excluding EDoS. The presence of unbalanced data has the potential to result in model overfitting [14]. Thus, we used the SMOTE-ENN (Synthetic Minority Oversampling Technique with the Edited Nearest Neighbors) balance technique [14], which combines over- and under-sampling, removing noisy and irrelevant samples from the dataset. SMOTE-ENN balanced the dataset with 151,567 and 145,603 samples with and without attacks, respectively.

Finally, in the last preprocessing step, the balanced dataset was split into 70% of the data for training/testing and the remaining 30% for prediction.

C. Algorithms used for hyperparameters definition

Nature-inspired algorithms have been used to define the hyperparameters of RF and XGB, which are then used to detect EDoS attacks. All nature-inspired algorithms were applied with default settings adopted in Python Sklearn Nature-inspired algorithms based on Niapy micro framework [30].

We considered the following algorithms: Bat algorithm (BA) [28]; Hybrid bat algorithm (HBA) [8]; Hybrid self-adaptable bat algorithm (HSABA) [9]; Firefly algorithm (FA) [29]; Grey wolf optimizer algorithm (GWO) [17]; and Genetic algorithms (GA) [11].

As already commented, we investigated how the previously listed nature-inspired algorithms can be used to define the hyperparameters of RF [16] and XGB [6], and also compared the results to solutions that use Random Search (RS) and Bayesian Search (BS) techniques.

D. Metrics

We consider two types of metrics to evaluate the algorithms. Firstly, we describe the usual metrics defined to evaluate the performance of the solutions. After, we also present metrics that consider the computational costs (CPU and RAM) and the time demanded to execute the algorithms.

1) Performance metrics: Our performance evaluation involves the utilization of widely recognized metrics such as accuracy, precision, recall, error rate, and F1-score. These metrics are derived from the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) that occur during the prediction process.

- Accuracy - quantifies the percentage of predictions correct by the model compared to the total number of predictions made, defined as

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad \text{(III.2)}$$

- Precision - measures the proportion of positive hits of the model, being crucial in situations where false positives are costly. The $Precision$ is defined as

$$Precision = \frac{TP}{TP + FP}. \quad \text{(III.3)}$$

- Recall - also known as sensitivity, it is an evaluation metric that evaluates the model's ability to correctly identify all positive examples in a data set. In simple terms, recall calculates the percentage of true positives in relation to the total number of examples that are actually positive. $Recall$ is defined as

$$Recall = \frac{TP}{TP + FN}. \quad \text{(III.4)}$$

- F1-score - is an evaluation metric that summarizes precision and recall in a single value. It is particularly useful when seeking an ideal balance between precision and recall. The $F1$ score is defined as

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall}. \quad \text{(III.5)}$$

- Error Rate - The proportion of incorrect predictions made by a classification model over the total number of predictions. $ErrorRate$ is defined as:

$$ErrorRate = \frac{\text{Number of Incorrect Predictions}}{\text{Total Number of Predictions}}. \quad \text{(III.6)}$$

2) Computational costs, search duration, and detection time metrics: While the measures described above are frequently employed to assess model effectiveness, there is currently no universally accepted benchmark for assessing computational costs. Therefore, we also assessed CPU usage, RAM consumption, hyperparameter search duration in the training phase (time required to optimize the hyperparameters for RF and XGB classifiers), and detection time.

IV. Experiments and Discussion

This section reports on the experimental evaluation of the discussed algorithms, aiming at answering the following main questions: *Can nature-inspired metaheuristic algorithms optimize hyperparameters in RF and XGB models to improve the detection of EDoS attacks? How do these algorithms perform in comparison to classic random and Bayesian search methods? What are the computational costs associated with each solution, including CPU, RAM, search duration, and detection time?* Overall, the experimental findings indicate that the combination of the BA algorithm with XGBoost performed better than the other combinations.

A. Experimental Setup

In our experimental study, we used a *GNU/Linux Ubuntu 20.04* operating system with *Intel Xeon CPU* with 16 cores and 24 *GB* of *RAM*. We also used *scikit-learn 1.3.0* for algorithms *RF, XGB, Random Search and Bayesian Search*, and the library *Sklearn Nature-inspired algorithms* [30], for the nature-based metaheuristic algorithms and *NatureInspiredSearchCV*, which were configured on the Anaconda platform version 2.4.

TABLE I: Random Forest algorithm parameters

Optimization Parameters (param_rf)	(a) *bootstrap* = {*True, False*} (b) *criterion* = {*gini, entropy*} (c) *max_depth* = {*80, 90, 100, 110*} (d) *min_samples_leaf* = {*3, 4, 5*} (e) *min_samples_split* = {*8, 10, 12*} (f) *n_estimators* = {*100, 200, 300, 1000*} (g) *max_features* = {*auto, sqrt, log2*}

TABLE II: XGboost algorithm parameters

Optimization Parameters (param_xgb)	(h) *gamma* = {*0, 0.5, 1*} (i) *n_estimators* = {*100, 200, 500*} (j) *max_depth* = {*5, 7, 10*} (k) *tree_method* = {*'auto', 'exact', 'approx', 'hist'*} (l) *learning_rate* = {*0.01, 0.05, 0.1*} (m) *reg_alpha* = {*0, 0.5, 1*} (n) *reg_lambda* = {*0, 0.5, 1*} (o) *booster* = {*gbtree*} (p) *objective* ={*binary:logistic*}

B. Experiment Methodology

For the experiments, the UNSW-NB15 database with EDoS attacks is first loaded and preprocessed as described in Section III. Then, the hyperparameters that must be optimized for the classifier algorithms RF [26] and XGB [5], also called *param_rf* and *param_xgb*, respectively, are also loaded.

Table I presents the RF parameters and their respective ranges. The optimization algorithm must select one of these values according to its optimization rules. The parameter *bootstrap* is a statistical resampling technique that takes the values 'True' or 'False' to assess how accurate the estimates are on the dataset. The *criterion* could be optimized as *gini* or *entropy*, where gini (gini impurity) is used to isolate in a branch the records that represent the most frequent class, while in entropy the balancing of the number of records in each branch is performed. The *max_depth* is the maximum depth for the decision tree. Large depth values can lead to overfitting while a value that is too small can cause underfitting. The *min_samples_leaf* defines the minimum value of samples needed to form a leaf node of the decision tree. The *min_sample_split* represents the minimum number of samples needed to split a node in the decision tree. The *n_estimator* is used to define the maximum number of trees in the forest. A large number usually provides better prediction while making the model take longer to converge. Finally, *max_features* defines the maximum number of features the model can use to split each node of the decision tree.

The XGboost algorithm parameters are presented in Table II. The parameter *gamma* determines the minimum loss for a new partition on a leaf node. The parameter *tree_method* determines the method used to build the tree in the XGB algorithm, it can be configured with the following values: *auto* - which uses heuristics to choose the fastest method, *exact*

TABLE III: NatureInspiredSearchCV Settings

param	*param_rf, param_xgb*
clf	*RandomForestClassifier, XGBoostClassifier*
cv	*5*
Algorithm	*BA, HBA, HSABA, FA, GWO, GA*
population_size	*50*
max_n_gen	*100*
max_stagnating_gen	*5*
runs	*5*
n_jobs	*-1*

TABLE IV: Random Search and Bayesian Search Settings

hyper	*param_rf, param_xgb*
cv	*5*
n_jobs	*-1*
clf	*RandomForestClassifier, XGBoostClassifier*
refit	*True*

- an exact greedy algorithm, *approx* - approximate greedy algorithm that uses quantile sketch and gradient histogram, and *hist* - agile algorithm optimized for the greedy approximate histogram. The parameter *learning_rate* controls the step size at which the optimizer can make updates to the weights. The parameters *reg_alpha*, called L1 regularization term, and *reg_lambda*, called L2 regularization term, contribute to reducing overfitting by adding a penalty term to the loss function. The parameter *booster* defines the tree model, while *objective* establishes the learning objective, where we choose *binary:logistic* for logistic regression which is used for binary classification. Finally, *n_estimators* and *max_depth* are defined as in RF.

In the next step, we apply the Nature Inspired Search, Random Search, or Bayesian Search methods to optimize the parameters we mentioned previously. Tables III and IV show the parameters used on each approach. The nature-inspired algorithms in the Sklearn package have been implemented using their default parameters. It is important to note that we define the parameter *param*, which contains the classifier parameters (e.g., *param_rf* or *param_xgb*) and in the *clf* we configure the classifier algorithm (e.g., *RandomForestClassifier* or *XGBoostClassifier*). Cross-validation (*cv*) was configured with five executions (k-fold), and *Algorithm* was used to choose the metaheuristic algorithm. We set the population size to 50 (*population_size*), the maximum number of generations to 100 (*max_n_gen*), and the *max_stagnating_gen* to 5. If the candidate search remains unaltered for five generations, the optimization process must be halted for this particular execution. The parameter *n_jobs* has been set to "-1", which means that all processors will be used. In addition, we also use the default Random Search and Bayesian Search parameters from the Sklearn library. In both cases, we configure the *hyper* to load the classifier model parameters (e.g., *param_rf* or *param_xgb*) and the *refit* to refit an estimator using the best parameters found in the entire dataset. The parameters *cv, clf* and *n_jobs* are defined as in the nature-inspired algorithms.

Finally, in the last stage, once RF or XGB have been configured with the hyperparameters optimized in the preceding step, the EDoS attack prediction is carried out and the results are gathered. Each configuration was executed five times, and we present the average of the values obtained from these executions, along with a 95% confidence interval.

C. Experimental Results

This section discusses the experimental results. We start by presenting the hyperparameters optimized by each solution in Table V. It is important to note that each search model selected a different set of parameters for RF, while most algorithms selected the same optimizations for XGB. Although both RF and XGB algorithms are capable of dealing with small variations in hyperparameters, substantial modifications can lead to markedly distinct outcomes. In our experiments, this aspect is exacerbated in the XGB optimized with Random Search, which selected different values for some parameters, leading to a solution with lower performance in terms of accuracy, prediction, F1-Score and error rate.

Table VI presents the performance results, computational costs, search duration and detection time. For each metric, we present the average value (*avg*) and its standard deviation (*sd*), highlighting the solution with the best result. Additionally, Figure 2 shows the average values for the metrics related to the training phase, i.e., the time demanded do optimize the hyperparameters (search duration) and the percentage of RAM consumption and CPU usage during this processing. On the other hand, Figure 3 displays the average values for the metrics related to the detection phase, i.e., the time demanded to detect an attack (notice we report the time spent to process all the 30% of the dataset used in this phase), and the performance metrics (accuracy, precision, recall, F1-Score and error rate).

TABLE V: RF and XGB hyperparameters, identified with the letters defined in tables I and II, selected by each solution

Algorithm	**Random Forest (RF)**							**XGBoost (XGB)**								
	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)	(m)	(n)	(o)	(p)
BA	False	Gini	110	3	8	300	log2	0	500	10	exact	0.1	0	0	gbtree	binary:logistic
HSABA	False	entropy	110	3	8	100	sqrt	0	500	10	exact	0.1	0	0	gbtree	binary:logistic
HBA	False	log_loss	80	3	8	100	sqrt	0	500	10	exact	0.1	0	0	gbtree	binary:logistic
FA	False	entropy	100	3	8	200	sqrt	0	500	10	exact	0.1	0	0	gbtree	binary:logistic
GWO	False	log_loss	90	3	8	200	log2	0	500	10	exact	0.1	0	0	gbtree	binary:logistic
GA	False	log_loss	100	3	8	1000	sqrt	1	500	10	exact	0.1	0	0	gbtree	binary:logistic
BS	False	Gini	90	3	8	300	log2	0	500	10	exact	0.1	0	0	gbtree	binary:logistic
RS	False	Gini	100	3	10	200	sqrt	1	500	7	exact	0.01	0	0	gbtree	binary:logistic

TABLE VI: Final Results

Algorithm	Training						Detection											
	Search Duration (s)		CPU (%)		RAM (%)		Accuracy (%)		Precision (%)		Recall (%)		F1-score (%)		Error Rate (%)		Detection Time (s)	
	avg	sd	avg	sd	avg	sd	avg	sd	avg	sd	avg	sd	avg	sd	avg	sd	avg	sd
BA + RF	5.84×10^{4}	3.18×10^{3}	56.49	3.58	**41.81**	1.25	99.37	0.02	99.15	0.05	99.56	0.02	99.35	0.02	0.62	0.02	1.57×10^{2}	3.70×10^{1}
BA + XGB	3.53×10^{3}	3.47×10^{2}	34.73	3.28	45.38	1.68	**99.53**	0.02	99.39	0.02	**99.65**	0.03	**99.52**	0.02	**0.46**	0.02	$\mathbf{9.60\times10^{1}}$	1.20×10^{1}
HSABA + RF	2.13×10^{5}	4.37×10^{3}	41.52	3.29	55.62	1.46	99.39	0.02	99.20	0.06	99.56	0.04	99.38	0.02	0.60	0.02	3.40×10^{2}	2.90×10^{1}
HSABA + XGB	1.88×10^{4}	5.08×10^{2}	36.26	3.35	59.12	1.57	99.50	0.04	99.40	0.03	99.57	0.07	99.49	0.05	0.49	0.04	1.35×10^{2}	6.00×10^{0}
HBA + RF	9.07×10^{4}	5.63×10^{3}	40.15	4.92	46.57	1.89	99.39	0.02	99.20	0.06	99.56	0.04	99.38	0.02	0.60	0.02	1.38×10^{2}	2.50×10^{1}
HBA + XGB	8.80×10^{3}	1.20×10^{2}	34.43	3.67	51.63	1.38	99.50	0.04	99.40	0.03	99.57	0.07	99.49	0.05	0.49	0.04	1.02×10^{2}	1.57×10^{2}
FA + RF	2.08×10^{5}	9.73×10^{3}	43.76	2.51	56.64	1.27	99.39	0.02	99.20	0.06	99.56	0.04	99.38	0.02	0.60	0.02	3.38×10^{2}	3.10×10^{1}
FA + XGB	2.69×10^{4}	5.43×10^{2}	**34.31**	3.20	57.17	1.03	99.50	0.04	99.40	0.03	99.57	0.07	99.49	0.05	0.49	0.04	1.61×10^{2}	1.30×10^{1}
GWO + RF	1.04×10^{5}	3.66×10^{3}	43.08	3.25	57.00	1.33	99.39	0.02	99.20	0.06	99.56	0.04	99.38	0.02	0.60	0.02	2.50×10^{2}	6.00×10^{0}
GWO + XGB	1.43×10^{4}	5.28×10^{2}	46.15	3.68	57.59	1.58	99.50	0.04	99.40	0.03	99.57	0.07	99.49	0.05	0.49	0.04	1.46×10^{2}	1.40×10^{1}
GA + RF	3.49×10^{5}	3.43×10^{4}	100.00	4.21	48.26	1.39	99.38	0.03	99.19	0.06	99.54	0.05	99.37	0.03	0.61	0.03	7.80×10^{2}	1.42×10^{2}
GA + XGB	4.36×10^{4}	2.46×10^{3}	100.00	3.95	44.02	1.63	99.50	0.04	99.40	0.03	99.57	0.07	99.49	0.05	0.49	0.04	5.30×10^{2}	7.40×10^{1}
BS + RF	8.03×10^{3}	5.90×10^{2}	35.18	3.39	61.51	2.25	99.52	0.01	99.40	0.02	99.62	0.02	99.51	0.01	0.47	0.01	1.07×10^{3}	1.88×10^{2}
BS + XGB	1.72×10^{3}	9.10×10^{1}	35.31	6.35	46.04	2.41	99.39	0.05	99.22	0.09	99.53	0.05	99.37	0.05	0.60	0.05	1.18×10^{2}	1.40×10^{1}
RS + RF	2.80×10^{3}	1.35×10^{2}	47.46	6.84	59.93	2.16	99.52	0.01	**99.41**	0.04	99.61	0.05	99.51	0.01	0.47	0.01	9.70×10^{1}	5.20×10^{1}
RS + XGB	$\mathbf{4.84\times10^{2}}$	3.86×10^{2}	38.94	6.82	51.78	1.18	94.37	0.09	94.92	0.12	93.44	0.75	94.17	0.02	5.62	0.09	9.90×10^{1}	7.00×10^{0}

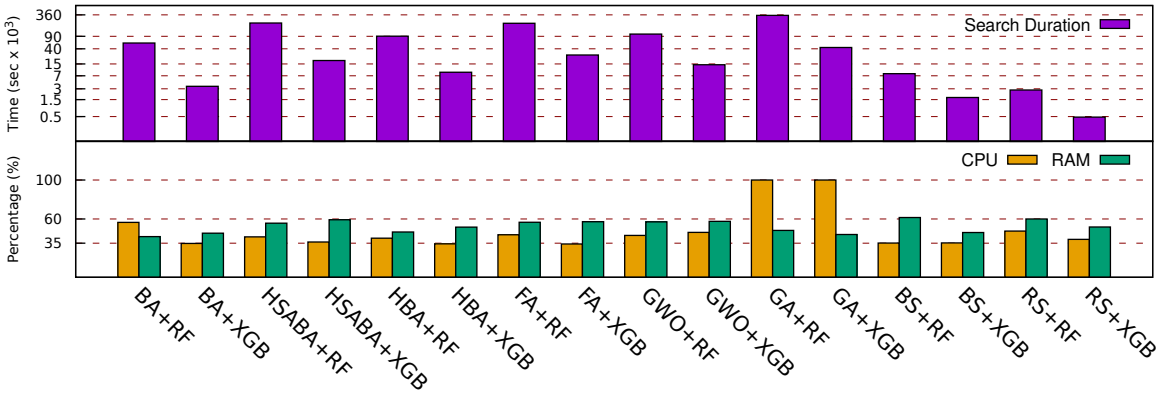

Fig. 2: Training results: search duration (top); CPU and RAM usage (bottom)

In general, BA+XGB outperformed other combinations (except for precision), with an F1-Score of 99.52%. This metric is important since it represents a balance between precision and recall. BA+XGB also presented an accuracy of 99.53%, precision of 99.39%, recall of 99.65%, and error rate of 0.46%. Moreover, this combination presented a search duration competitive with the others, except for RS+XGB. The CPU usage and RAM consumption for BA+XGB also are similar to the combinations that better performed, i.e., FA+XGB and BA+RF for CPU usage and RAM consumption, respectively.

The RS+XBG algorithm achieved the best result for search duration with 4.84×10^{2} seconds, this means that the traditional RS-optimized XGB is faster than the other combinations. However, at the same time, this combination presented the worst performance (F1-Score is only 94.17%, accuracy of 94.37%, precision of 94.92%, recall of 93.44% and error rate of 5.62%). This behavior is mainly justified by the maximum depth for the decision tree that was optimized to only 7, while other algorithms optimized the decision tree of XGB with a maximum depth of 10 (Table V).

It is worth noting that XGB was optimized faster than RF among the studied methods. This aspect is especially relevant in scenarios where it is desirable to update the detection model. On the other hand, CPU usage and RAM consumption were similar for all combinations, except when using genetic algorithm (GA) which used all the CPU processing power (100%). These aspects are especially important in scenarios with limited available resources.

In summary, the experiments indicate that it is possible to use nature-inspired algorithms to optimize hyperparameters, especially when they are used in conjunction with the XGB to identify EDoS attacks. In general, these algorithms showed better performance than the RF algorithm. We highlight the bat-based algorithm (BA+XGB) due to its high performance, fast detection, low usage of computational resources, and good efficiency in optimizing hyperparameters. This makes it an interesting option for identifying EDoS attacks, especially in situations where traffic patterns change frequently and the detection model needs to be constantly updated.

V. Conclusion

Cloud computing, while cost-effective, faces security challenges such as EDoS attacks. Machine learning algorithms, such as Random Forest and XGBoost, can detect these attacks. However, the effectiveness of these algorithms depends on the proper selection of hyperparameters, obtained by methods such as Random Search or Bayesian Search. In our work, nature-inspired metaheuristic algorithms were used for optimization, which in general demanded fewer computational resources than Random Search and Bayesian Search. Among the combinations experimented, BA+XGB best performed. This work

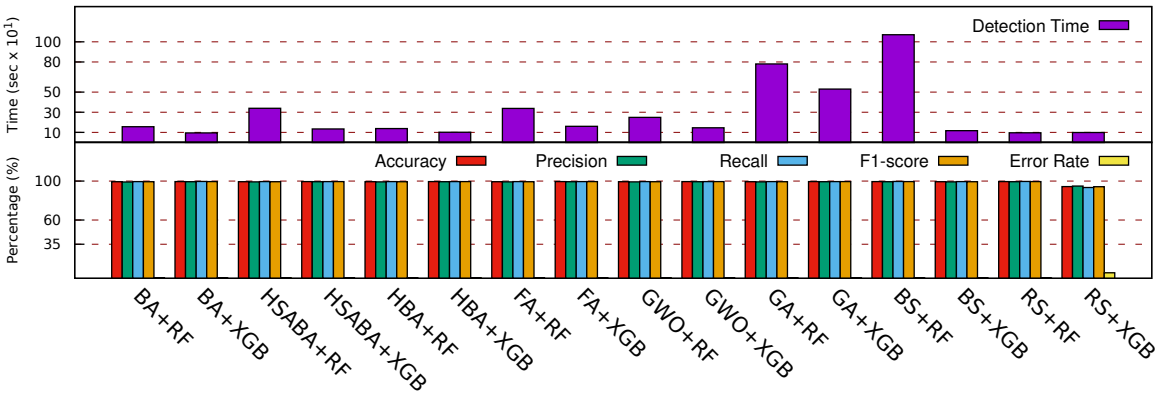

Fig. 3: Detection results: detection time (top); accuracy, precision, recall, F1-score and error rate (bottom)

aims to assist in choosing solutions for EDoS attack detection, balancing performance, cost, and detection time.

ACKNOWLEDGEMENTS

This work was partially funded by FAPDF through Notice 04/2021 and by CAPES-Brazil through process 88887.978125/2024-00.

REFERENCES

[1] N. Agrawal and S. Tapaswi, "A proactive defense method for the stealthy edos attacks in a cloud environment," *International Journal of Network Management*, vol. 30, no. 2, p. e2094, 2020.

[2] T. H. Aldhyani and H. Alkahtani, "Artificial intelligence algorithm-based economic denial of sustainability attack detection systems: Cloud computing environments," *Sensors*, vol. 22, no. 13, p. 4685, 2022.

[3] H. I. H. Alsaadi, M. K. Al-Anni, and F. E. K. Al-Khuzaie, "Deep learning to mitigate economic denial of sustainability (edos) attacks: Cloud computing," in *International Conference on Emerging Smart Technologies and Applications*, 2023, pp. 1–7.

[4] K. Bhushan and B. B. Gupta, "Network flow analysis for detection and mitigation of fraudulent resource consumption (frc) attacks in multimedia cloud computing," *Multimedia Tools and Applications*, vol. 78, no. 4, pp. 4267–4298, 2019.

[5] T. Chen and C. Guestrin, "extreme gradient boosting project," https://github.com/dmlc/xgboost, 2016, accessed on 08 July 2024.

[6] ——, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.

[7] A. C. de Lima, J. L. Bordim, and E. A. Alchieri, "Detecting edos attacks in cloud environments using machine learning and metaheuristic algorithms," in *Eleventh CANDAR Workshops*, 2023.

[8] I. Fister Jr, D. Fister, and X.-S. Yang, "A hybrid bat algorithm," *arXiv preprint arXiv:1303.6310*, 2013.

[9] I. Fister Jr, S. Fong, J. Brest, and I. Fister, "A novel hybrid self-adaptive bat algorithm," *The Scientific World Journal*, vol. 2014, 2014.

[10] D. G. Goswami, "Imbalanced dataset using smote," https://www.kaggle.com/code/devanggiri/imbalanced-dataset-using-smote/notebook/, 2024, accessed on 24 July 2024.

[11] J. H. Holland, "Genetic algorithms," *Scientific american*, vol. 267, no. 1, pp. 66–73, 1992.

[12] M. S. Hossain and M. S. Islam, "Economic denial of sustainability attack detection using machine learning," in *2023 26th International Conference on Computer and Information Technology*, 2023.

[13] J. Idziorek and M. Tannian, "Exploiting cloud utility models for profit and ruin," in *International Conference on Cloud Computing*, 2011.

[14] I. Jung, J. Ji, and C. Cho, "Emsm: ensemble mixed sampling method for classifying imbalanced intrusion detection data," *Electronics*, vol. 11, no. 9, p. 1346, 2022.

[15] Lima, Alessandro, "Database edos-unsw-nb15," https://github.com/aclalessandro/EDoS-UNSW-NB15, 2024, accessed on 30 may 2024.

[16] T. J. Lucas, I. S. De Figueiredo, C. A. C. Tojeiro, A. M. G. De Almeida, R. Scherer, J. R. F. Brega, J. P. Papa, and K. A. P. Da Costa, "A comprehensive survey on ensemble learning-based intrusion detection approaches in computer networks," *IEEE Access*, 2023.

[17] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in engineering software*, vol. 69, pp. 46–61, 2014.

[18] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *Military Communications and Information Systems Conference*, 2015.

[19] L. Munson, "Greatfire. org faces daily $30,000 bill from ddos attack," *Org Faces Daily*, vol. 30000, 2015.

[20] M. Pocwierz, "How an empty s3 bucket can make your aws bill explode," https://medium.com/@maciej.pocwierz/how-an-empty-s3-bucket-can-make-your-aws-bill-explode-934a383cb8b1, 2024.

[21] M. Rambabu, S. Gupta, and R. S. Singh, "Data mining in cloud computing: survey," in *Innovations in Computational Intelligence and Computer Vision: Proceedings of ICICV 2020*, 2021, pp. 48–56.

[22] D. K. K. Reddy, J. Nayak, and M. Mishra, "Intrusion detection system in iot smart city environment using tree-based approach with swarm-based optimization for multi-step cyber-attack dataset," in *1st International Conference on Circuits, Power and Intelligent Systems*, 2023.

[23] S. Ribin Jones and N. Kumar, "An efficient edos-dome system in cloud computing using obfuscated ip spoofing technique and rcdh-enn detection technique," *Applied Nanoscience*, pp. 1–13, 2021.

[24] S. M. Rubai, "Development of hyper-parameter-tuned-recurrent neural network for detection and mitigation of fraudulent resource consumption attack in cloud," *Transactions on Emerging Telecommunications Technologies*, vol. 34, no. 3, p. e4723, 2023.

[25] S. Q. A. Shah, F. Z. Khan, and M. Ahmad, "The impact and mitigation of icmp based economic denial of sustainability attack in cloud computing environment using software defined network," *Computer Networks*, vol. 187, p. 107825, 2021.

[26] sklearn. ensemble. RandomForestClassifier—, "scikit-learn 0.24. 2 documentation," accessed on 27 July 2023.

[27] M. Suguna, M. Mohan, J. Srikanth, M. Suresh, P. Banupriya, and L. Dhavamani, "Detection of edos attacks in sdn-based cloud model using deep learning based sdpn technique," in *Third Int. Conference on Smart Technologies in Computing, Electrical and Electronics*, 2022.

[28] X.-S. Yang, "A new metaheuristic bat-inspired algorithm," in *Nature inspired cooperative strategies for optimization*, 2010, pp. 65–74.

[29] X.-S. Yang and X. He, "Firefly algorithm: recent advances and applications," *International journal of swarm intelligence*, vol. 1, no. 1, pp. 36–50, 2013.

[30] Zaťko, Timotej and Fister Jr., Iztok and Sinha, Ankur, "Nature-inspired algorithms for scikit-learn," https://github.com/timzatko/Sklearn-Nature-Inspired-Algorithms/tree/master, 2020, accessed on 27 July 2023.

[31] W. Zhijun, L. Wenjing, L. Liang, and Y. Meng, "Low-rate dos attacks, detection, defense, and challenges: a survey," *IEEE access*, vol. 8, pp. 43 920–43 943, 2020.